An Open-Source Plug-and-Play Testbed for SDN Controller Performance Benchmarking

Tamerlan Abdullayev
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abdulltn@mail.uc.edu

Giovani Abuaitah
Department of Computer Science
University of Cincinnati
Cincinnati, OH, USA
abuaitgi@ucmail.uc.edu

***Abstract*—Software-Defined Networking (SDN) moves forwarding decisions from individual switches to a central software controller, so the controller's speed directly affects network performance. Measuring that speed reliably is harder than it seems. Existing benchmarking tools test controllers with artificial request streams rather than real network traffic, while studies that emulate real networks are usually run by hand, rarely repeat their experiments, and seldom report how much results vary between runs, so their findings are difficult to reproduce or compare. We present an open source testbed that addresses this gap by automating the full benchmarking process. It emulates a network of switches and hosts, sends real TCP traffic between the hosts, and records how long the controller takes to handle each request a switch sends it about a new flow. Every experiment is driven by a random seed, so its traffic can be regenerated exactly; each configuration is repeated to quantify variability; and every run is checked to confirm that it delivered its configured traffic, so that results distorted by an overloaded emulation machine are detected rather than reported. Because the testbed records every individual response time, it shows not only how fast a controller typically responds but also how slow its slowest responses become, which averages and round trip delay measurements hide. New controllers and network layouts can be added through small, documented modules without changing the rest of the testbed.**

***Index Terms*—SDN, benchmarking, testbed, reproducibility, network emulation, OpenFlow, controller performance, open source.**

I. Introduction

Software-defined networking (SDN) [1] decouples the control plane from the data plane, placing forwarding intelligence in a software controller. As SDN deployments scale [2], rigorous and reproducible benchmarking of controller performance becomes increasingly important for SDN research, controller selection, scalability assessment, and capacity planning.

The Reproducibility Gap. Despite the availability of several benchmarking tools, the SDN research community lacks an open-source, end-to-end testbed that is both systematic and reproducible across studies. Synthetic tools such as CBench [3] emulate switches that send Packet-In messages to the controller but generate no end-to-end TCP flows over an emulated topology. RFC 8456 [4] recommends evaluation across multiple topologies and network sizes with repeated trials, but, to the best of our knowledge, open-source testbeds that implement these recommendations in an automated, reusable manner remain limited. Emulation studies using Mininet and iPerf [5]–[7] typically measure a single host pair per scenario and report neither run-to-run variability nor per-event controller processing time, which makes their findings challenging to verify or reuse.

Our Contribution. We present an open-source, plug-and-play SDN benchmarking testbed[1] that addresses these gaps. Its contributions are: (i) a seed-controlled, fully automated pipeline that orchestrates experiment execution, network emulation, traffic generation, result aggregation, and cleanup, in which the traffic schedule of every run can be regenerated from its seed, repetitions with distinct seeds quantify run-to-run variability, and full sweeps run without manual intervention; (ii) per-event controller instrumentation that records the processing time of every Packet-In handler invocation with a high-resolution monotonic clock, enabling separate characterization of median (p50) and tail (p95) controller-side latency, a distinction not possible with the RTT-based measurements used in prior work; (iii) end-to-end TCP traffic with verified delivery, in which rate-paced TCP flows are launched at a configurable rate between randomly selected host pairs, so that Packet-In events originate from real host-to-host connections across multiple switches rather than from synthetic stimulus, and every run reports its flow success rate and achieved flow rate, so that results obtained under failed or delayed traffic generation can be detected rather than silently aggregated; and (iv) plug-and-play extensibility, with flexible configuration of network topologies and traffic patterns, new topologies added as modules, and new controllers integrated through a documented adapter contract (Section III-E) without modifying the rest of the pipeline, as demonstrated with Ryu, OS-Ken, and ONOS.

II. Background and Related Work

A. SDN and OpenFlow

In SDN, the forwarding devices, called *switches* in OpenFlow whether they forward at layer 2 or layer 3, hold flow tables of rules that map packets to forwarding actions, and a software controller decides which rules to install. The OpenFlow protocol [1], [8] provides a standard communication

[1]The testbed is available at https://github.com/tamerlanabdullayev/sdn-testbed.

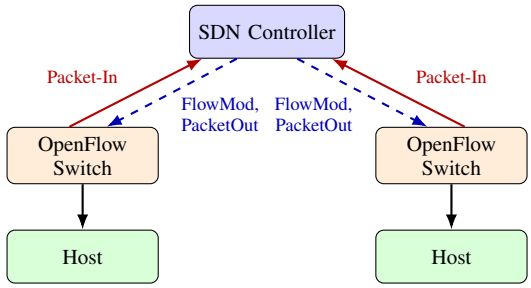


Fig. 1. SDN control interactions over OpenFlow: a switch sends a Packet-In to the controller for a packet that matches no rule, and the controller replies with a FlowMod and a PacketOut.

interface between the controller and the switches. When a packet matches no rule in a switch's flow table, the switch sends a *Packet-In* message to the controller, which replies with a *FlowMod* that installs a rule for the flow and a *PacketOut* that tells the switch how to forward the packet (Fig. 1). Later packets of the same flow are forwarded by the switch without involving the controller, so the time the controller needs to handle each Packet-In determines how quickly new flows are set up.

B. Synthetic Benchmarking Tools

The most widely used tool for SDN controller benchmarking is CBench [3], which emulates a set of OpenFlow switches that send Packet-In messages to the controller and measures the rate of returned FlowMod responses. While widely cited, CBench emulates switches without modeling inter-switch connectivity, supports only OpenFlow 1.0 [9], and does not generate end-to-end TCP flows between hosts; it therefore may not fully capture the topology-driven event dynamics that arise in realistic deployments. Zhu et al. [9] qualitatively classified 34 controllers and benchmarked nine of them using CBench, PktBlaster, and OFNet, noting that *none of the tools available can measure all performance statistics* and that performance metrics are often simplified or interpreted inconsistently across tools and prior studies. OFLOPS [10] targets switch rather than controller evaluation, while WCBench [11] (tied to a specific OpenDaylight release) and HCprobe [12] extend CBench but retain its synthetic traffic model.

C. IETF Benchmarking Standards and Gaps

The IETF Benchmarking Methodology Working Group (BMWG) has published several relevant informational RFCs. RFC 8456 [4] defines a methodology for benchmarking SDN controller performance, recommending that controllers be evaluated across multiple network topologies and a varying number of network devices, and that each test be repeated a minimum of 10 times to increase confidence in the results, with averages and variances reported across trials. RFC 5180 [13] notes that the device under test can be monitored for processor and memory utilization. Our testbed draws on both, with multi-topology sweeps of 10 repetitions per configuration and per-second controller CPU and memory monitoring. These documents, however, address methodology only and do not prescribe open-source tooling or reference implementations.

TABLE I
COMPARISON OF SDN BENCHMARKING APPROACHES

Feature	CBench [3]	Mininet studies†	Our Testbed
Emulated inter-switch topology	✗	✓	✓
End-to-end TCP traffic	✗	✓	✓
Traffic delivery validation	✗	✗	✓
Automated end-to-end pipeline	○	✗	✓
Factorial parameter sweep	○	○	✓
Repeated trials w/ variability	✓	○	✓
Per-event controller latency	✗	✗	✓
CPU/memory monitoring	✗	✗	✓
Public source code	✓	✗	✓
Pluggable topology module	✗	✗	✓

✓ = supported; ○ = partial; ✗ = absent or not reported. † [5]–[7].

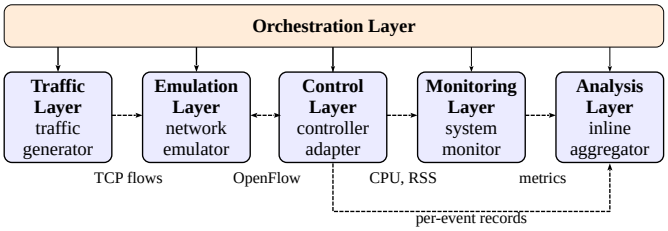


Fig. 2. Testbed architecture. The orchestration layer coordinates all layers; the analysis layer consolidates the outputs of each run.

D. Academic Studies Using Mininet and iPerf

Several recent studies use Mininet [14] and iPerf but observe controller behavior primarily through end-to-end metrics. Koulouras et al. [5] compared four controllers on k-ary fat-trees (k=4–10), measuring throughput, RTT, and jitter between one host pair per scenario, without reporting repeated trials. Čabarkapa et al. [6] evaluated Ryu and ONOS on a single fat-tree (32 hosts) between one host pair, inspecting OpenFlow message rates with Wireshark, but did not vary network size or load. Ali [7] compared four controllers across five topologies using 50 iPerf and ping packets per configuration, without reporting variability. None of these studies report a public code release. Table I summarizes the methodological gaps addressed by our testbed.

III. TESTBED DESIGN

A. Overview

Fig. 2 illustrates the testbed architecture. An orchestration layer coordinates five layers: the *traffic layer*, which generates the workload; the *emulation layer*, which emulates the network; the *control layer*, which runs the controller under test through an adapter; the *monitoring layer*, which samples the controller's resource use; and the *analysis layer*, which aggregates the outputs of each run. Each layer is defined by the interface it exposes rather than by a specific tool, so any of them can be replaced without modifying the others. The following subsections describe the layers in this order, starting with orchestration.

B. Orchestration Layer

For each run, the orchestration layer cleans residual emulator state, starts the controller through its adapter and waits until it is ready, starts the system monitor, runs a short warm-up network whose events are excluded from the results, launches the emulated network and the traffic schedule, waits for all flows to complete, stops all components, aggregates the run, and prunes bulky raw outputs once aggregation has succeeded. The seed of each run is derived deterministically from its parameters, so any individual run can be regenerated from its parameter tuple, and full factorial sweeps over all parameter combinations run unattended.

C. Traffic Layer: Traffic Model

Flows are launched at a configured rate (FPS) between pairs of distinct hosts chosen uniformly at random from a seeded generator, so the traffic schedule of a run depends only on its seed. Each flow is a TCP transfer of fixed duration d, paced to a fixed per-flow rate. Fixing the per-flow rate makes the offered load depend on the number of flows in flight (FPS $\times$ d) rather than on the host count, and keeps traffic generation from contending with the controller for CPU on a single emulation host. Every destination provides enough receivers that concurrent flows are not rejected. For each run, the traffic generator records the achieved launch rate, and every flow's outcome is classified, so that runs whose workload was not delivered as configured can be detected rather than silently aggregated.

D. Emulation Layer: Topology Module

Topologies are modules that expose a single builder parameterized by the host count N and are registered by name, so a new topology can be added without modifying the rest of the pipeline. Static address resolution is configured on all hosts, so no ARP traffic reaches the controller. The testbed currently includes two loop-free topologies used in this paper: a star (N hosts on a single switch) and a two-tier tree, in which one core switch connects $\lceil N/4 \rceil$ edge switches with up to four hosts each. An experimental full-mesh topology is also included but requires a loop-prevention mechanism (e.g., STP) in the control application and is not evaluated here.

E. Control Layer: Controller Adapter Contract

The testbed never interacts with a controller directly. Instead, each controller is wrapped in an *adapter*: a small module, specific to that controller, that translates the testbed's generic requests (start, stop, report results) into the commands and files of that controller. The *adapter contract* is the fixed set of obligations every adapter must meet, so that the orchestration, monitoring, and aggregation work unchanged for any controller that satisfies it. The contract has three parts. First, *lifecycle control*: the adapter starts the controller on a given OpenFlow port with a given per-run output directory, signals when the controller is ready to accept switch connections, reports the identifier of the controller process for monitoring, and stops it cleanly. Second, a *reference control application* that defines the workload: MAC-learning L2 switching in which flow entries match on the input port and the source and destination MAC addresses. Flow entries are installed without idle or hard timeouts, so each directed host pair triggers Packet-In events at a given switch only until its entry is installed, after which flows between that pair are forwarded without controller involvement. Third, *instrumentation* that emits two records in fixed schemas:

- a *per-event record* for every Packet-In, containing the switch identifier, source and destination addresses, and the *handler processing time*: the elapsed time from the moment the controller application begins handling the Packet-In until it has issued its response—a FlowMod followed by a PacketOut for a known destination, or a flooding PacketOut otherwise; and
- a *per-second record* of Packet-In and FlowMod counts, produced without blocking the controller's event loop.

The handler processing time isolates controller-side processing from network and switch delays, but excludes the time a message spends in transit and waiting in the controller's event queue before dispatch; queue residence time is a complementary metric that can be obtained without controller changes from a capture of the control channel, which the orchestrator records; analysing it is left to future work. Unlike the black-box measurement model of RFC 8456 [4], which observes timing at the test emulator interfaces, these timestamps are taken inside the controller application. Reporting both the median (p50) and 95th-percentile (p95) of these per-event times is important because the mean can mask tail behavior in event-driven systems under bursty load, as queueing analyses of SDN controllers [15] suggest.

Integrating a new controller therefore amounts to implementing the lifecycle operations and the reference application and emitting these two records; the emulator, traffic generator, monitor, orchestration, and aggregation remain unchanged.

F. Monitoring Layer: System Monitor

A separate process samples the controller's CPU utilization, resident memory (RSS), and thread count once per second. Running the monitor as a separate process keeps monitoring from interfering with the controller.

G. Analysis Layer: Metrics and Aggregation

After each run, the inline aggregator computes and appends to the results table: the *flow success rate* (fraction of planned flows completed without error) and *achieved flow rate* (flows launched divided by actual launch duration), which validate that the run delivered its configured workload; *measured throughput* (bytes sent by successful flows divided by run duration); the mean *Packet-In rate*; the p50 and p95 *handler processing time* over all Packet-In events, including flooding events; and mean controller *CPU* and *memory* over the traffic window, from the first flow launch until the last flow completes.

IV. Implementation

A. Network Emulation and Topologies

The network emulator is Mininet [14] with Open vSwitch [16] and TCLink links; static ARP entries are pre-populated with Mininet's `autoStaticArp` option. A topology is a Mininet `Topo` subclass with a single `build(self, n)` method, registered with one line.

B. Traffic Generation

Traffic is generated with iperf3 [17]. Because an iperf3 server serves one test at a time, each host runs a pool of $\lceil 1.5\lambda + 3\sqrt{\lambda}\,\rceil$ servers (bounded to 4–320), where $\lambda = \text{FPS} \cdot (d + 1\,\text{s})/N$ is the expected number of concurrent flows per destination, and each flow uses the least recently used port on its destination. Traffic starts once every server port is listening, launches follow a wall-clock schedule, pacing uses kernel socket pacing (`--fq-rate`), and each client's output is classified as successful, rejected, or failed.

C. Monitoring, Orchestration, and Aggregation

The system monitor uses `psutil` [18]. Orchestration is script-based: `run_experiment.py` takes the controller and run parameters as command-line arguments, pins the controller to a dedicated CPU core and the emulator and traffic generator to the remaining cores, detects readiness through the adapter (for example, by polling the controller's OpenFlow port), and records the control channel with `tcpdump`. Seeds are seed $= B_T + 10N + \text{FPS} + \text{rep}$, with $B_T = 1000$ for star and 2000 for the tree. A resumable sweep script runs all combinations, and the aggregator appends one row per run to a summary CSV.

V. Testbed Validation: Case Studies with Ryu, OS-Ken, and ONOS

A. Controller Adapters

The reference adapter is a Ryu [19] application (v4.34, OpenFlow 1.3, CPython 3.10) implementing the control logic and instrumentation of Section III. Each Packet-In handler invocation is timestamped on entry with `time.perf_counter()`, a high-resolution monotonic clock, and written in milliseconds to `flow_events.csv`; per-second counts are written by a background greenthread, so logging does not block the event loop, and the output directory is passed through an environment variable. Adapters for OS-Ken and ONOS are described in Section V-F.

B. Experimental Configuration

We run a full factorial experiment with the Ryu adapter over two topologies (star, two-tier tree), $N \in \{4, 8, 16, 32, 64\}$ hosts, and flow rates of 25, 50, 100, and 200 flows/s, with 2 s flows paced at 2 Mbit/s over 20 s runs and 10 repetitions with distinct seeds per configuration (400 runs), and repeat the identical sweep with the OS-Ken and ONOS adapters (1,200 runs in total). All experiments run on a single VirtualBox VM (Ubuntu 24.04, 16 vCPUs, 16 GB RAM); the controller is pinned to a dedicated vCPU, and the emulated network and traffic generator to three others. A run takes about 40 s with Ryu and OS-Ken and 70 s with ONOS; the full sweep completed unattended in 16.3 h, and a code hash recorded by every run confirms that all runs used identical code.

C. Workload Validation

Before controller metrics are analyzed, the testbed verifies that every run delivered its configured workload. Across all 1,200 runs, the flow success rate was at least 99.8% and the achieved flow rate was within 4% of the configured rate (within 0.2% for Ryu and OS-Ken). Measured throughput scales linearly with flow rate and differs by at most 1.2% between topologies, confirming equal offered load; as it uses iperf3's sender-side byte count, which includes data still queued when a flow ends, it is an upper bound, 27–30% above the nominal load. These checks matter: in preliminary runs with unpaced traffic and an unpinned controller, emulation-host saturation inflated the star's p95 handler processing time at $N{=}64$ to $\sim$18 ms, more than 40$\times$ the 0.40 ms measured in the final configuration—a distortion that controller-side metrics alone cannot distinguish from genuine controller behavior. Even with paced traffic, an unpinned controller showed a 30–50% higher p95 than a pinned one, because it competed with emulation processes for CPU.

D. Per-Event Processing Latency

Fig. 3 shows per-event handler processing time at p50 and p95 across flow rates at $N{=}64$. The p50 stays nearly constant at 0.13–0.16 ms for both topologies, so a median-based measurement would suggest that load has little effect. The p95, however, rises from 0.28 to 0.66 ms for the star and from 0.26 to 1.53 ms for the tree between 25 and 200 fps, with the steepest increase above 50 fps for the tree and above 100 fps for the star.

The sweep also exposes how workload parameters interact. Because Packet-Ins occur only on first contact between each directed host pair at each switch, at small N all pairs are discovered early in a run and flow rate has little effect (star, $N{=}4$: p95 of 0.41 and 0.43 ms at 25 and 200 fps); at $N{=}64$, many of the 4,032 directed pairs remain unseen even at 200 fps, so higher flow rates keep raising controller load. The $N{=}4$ case also shows that traffic volume alone does not inflate the tail, since as many flows are in flight as at $N{=}64$.

E. Scalability with Host Count

Fig. 4 and Table II show that p95 stays between 0.29 and 0.40 ms for both topologies up to $N{=}32$. At $N{=}64$, the tree's p95 roughly doubles to 0.74 ms, while the star's remains at 0.40 ms; the slightly higher p95 at $N{=}4$ reflects the few Packet-In events per run at this scale. Mean controller CPU during traffic rises from below 1% to 13% (star) and 20% (tree) and never exceeds 29% in any configuration, suggesting that the tail is not compute-bound.

The tree generates 1.7–1.8$\times$ more Packet-Ins than the star at the same flow rate, because a flow between hosts on different edge switches traverses three switches, each raising its own Packet-In. Although the tree spreads these over $\lceil N/4 \rceil + 1$

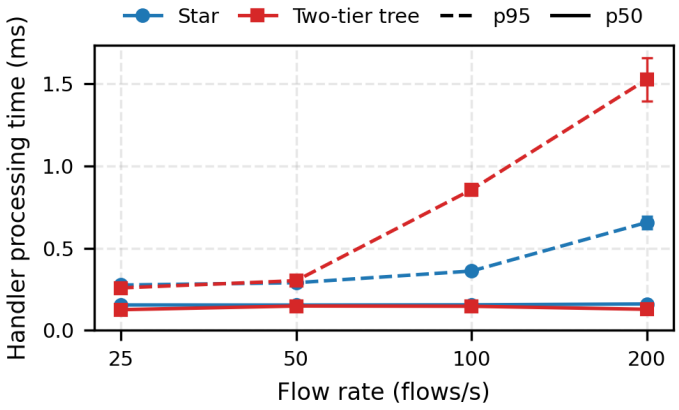


Fig. 3. Ryu, p50 (solid) and p95 (dashed) handler processing time vs. flow rate at N=64. Error bars show 95% confidence intervals of the mean over 10 repetitions.

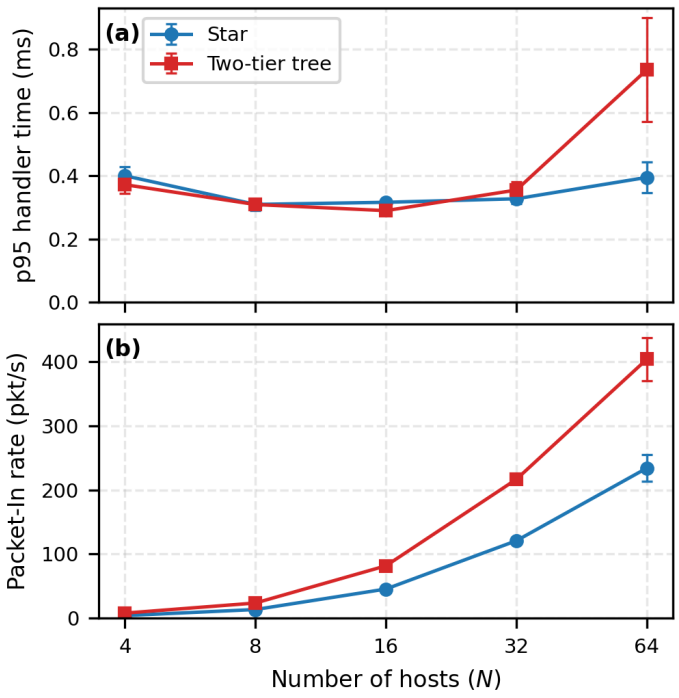


Fig. 4. Ryu, (a) p95 handler processing time and (b) Packet-In rate vs. host count, averaged over flow rates. Error bars show 95% confidence intervals of the mean. The tree's tail rises at N=64, while the star's stays nearly flat.

switches, its p95 at N=64 is 1.9× the star's, so per-switch concentration does not appear to be the primary explanation of the tail. The mean Packet-In rate alone does not explain it either: for the star at N=64, p95 rises by 83% from 100 to 200 fps while the mean Packet-In rate rises by only 7% (281 vs. 302 pkt/s), pointing to the burstiness of Packet-In arrivals, which the testbed's per-second counters can quantify.

F. Extensibility: OS-Ken and ONOS

Extending the testbed requires little code: the star topology module is 8 lines of Python and the two-tier tree under 20, and the Ryu adapter, including its instrumentation, is about 120 lines. To validate the testbed across controller architectures, rather than to compare controller performance, we integrated

TABLE II
RYU METRICS BY TOPOLOGY AND HOST COUNT (MEANS OVER FLOW RATES AND REPETITIONS; CPU OVER THE TRAFFIC WINDOW)

Topo.	N	Tput (Mbps)	Pkt-In (pkt/s)	p95 Lat (ms)	CPU (%)
Star	4	486	4	0.40	0.2
Star	8	487	13	0.31	0.6
Star	16	486	45	0.32	1.9
Star	32	482	121	0.33	6.1
Star	64	475	234	0.40	13.1
Tree	4	486	8	0.37	0.3
Tree	8	486	24	0.31	0.8
Tree	16	484	82	0.29	2.6
Tree	32	477	217	0.36	8.4
Tree	64	473	404	0.74	20.4

two further controllers and ran the identical 400-run sweep with each. OS-Ken, the actively maintained community fork of Ryu [19], required only seven `import` lines and a different base class in the Ryu application. ONOS [20] (v2.7.0) is architecturally different: a multithreaded Java controller that runs inside an OSGi container. Its adapter consists of a 199-line ONOS application that reimplements the reference control application and writes the same two records in the same schemas, 253 lines of Python and shell for lifecycle control and installation, and a 50-line build file. Apart from registering the adapter by name (8 lines), no shared component contains code specific to ONOS.

Integrating ONOS exposed two requirements that the original contract left implicit, and the pipeline now applies both to every controller: the adapter must signal readiness, because ONOS needs 20–28 s to accept switch connections compared with about 0.2 s for Ryu, and the monitored process must be the controller itself rather than its launcher script; a warm-up round also lets controllers that compile code at run time reach steady state. Two ONOS runs failed during startup, were caught by the readiness check, and were rerun, and the monitor recorded ONOS's very different footprint (about 1.5 GB of memory versus 60 MB for Ryu) without modification. Because ONOS installs flow rules through its own flow subsystem, its handler time covers different work than Ryu's, so we do not compare handler times across frameworks; the recorded control channel captures enable such comparisons and are left to future work.

Repeated runs are consistent: across the 40 Ryu configurations, the 95% confidence interval is within 10% of the mean for p50 in all 40 and for p95 in 28, with the widest intervals at small N.

VI. DISCUSSION

A. Lessons for Controller Benchmarking

Three lessons from the validation apply beyond the evaluated controllers. First, controller metrics are trustworthy only if the delivered workload is verified and the controller is isolated from the emulation: host saturation produced a tail inflation of more than 40× that resembled controller behavior, and even with paced traffic, sharing CPUs with the emulation

raised p95 by 30–50% (Section V-C). Second, the median hides load-dependent behavior that the per-event p95 exposes. Third, controller-side counters let explanations be tested: the data do not support per-switch concentration as the primary explanation of the tail and point to bursty Packet-In arrival.

B. Limitations of the Demonstration

All three controllers run the same simple L2 reference application, so the demonstration exercises the adapter contract and the measurement pipeline rather than richer control logic, and handler times are comparable only within a framework (Section V-F).

Emulation in a VM adds timing noise, so absolute values reflect the emulated environment. Without flow timeouts, Packet-In load is bounded by unique host pairs; flow-expiry workloads are future work. Paced 2 Mbit/s flows limit the offered data-plane load, and the evaluated tree lacks the redundant paths of a k-ary fat-tree, which require loop prevention in the control application.

VII. Conclusion

We presented an open-source, plug-and-play SDN controller benchmarking testbed. It contributes a seed-controlled automated pipeline, per-event controller instrumentation that separates p50 from p95 Packet-In handler processing time, rate-paced end-to-end TCP traffic with per-run validation of delivered flows and achieved flow rate, and a plug-and-play topology and controller adapter system. A validation study of 1,200 runs with Ryu, OS-Ken, and ONOS across star and two-tier tree topologies confirms that the testbed captures load-dependent controller behavior with quantified run-to-run variability—including a rise in tail latency at scale that median and aggregate RTT-only measurements do not reveal—and that the adapter contract accommodates both a closely related Python controller and an architecturally different Java controller without controller-specific changes to the shared pipeline.

The testbed is publicly available as open-source software; planned extensions include per-host containers, link-level parameters as first-class configuration, flow-expiry workloads, and a web-based interface.

References

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[3] Stanford University, "CBench: Controller benchmarking tool," https://github.com/mininet/oflops, 2012.

[4] V. Bhuvaneswaran, A. Basil, M. Tassinari, V. Manral, and S. Banks, "Benchmarking methodology for software-defined networking (SDN) controller performance," Internet Engineering Task Force, RFC 8456, 2018.

[5] I. Koulouras, S. V. Margariti, I. Bobotsaris, E. Stergiou, and C. Stylios, "On the performance of SDN controllers in real world topologies," in *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2022, pp. 143–148.

[6] D. Čabarkapa, D. Rančić, P. Pavlović, and M. Milićević, "Investigating the impact of tree-based network topology on the SDN controller performance," *Facta Universitatis, Series: Automatic Control and Robotics*, vol. 21, no. 1, pp. 25–35, 2022.

[7] U. Ali, "Performance comparison of SDN controllers in different network topologies," *Research Square (preprint)*, 2024, DOI: 10.21203/rs.3.rs-4826985/v1.

[8] Open Networking Foundation, "OpenFlow switch specification version 1.3.0," https://www.opennetworking.org, 2012.

[9] L. Zhu, M. M. Karim, K. Sharif, F. Li, X. Du, and M. Guizani, "SDN controllers: Benchmarking & performance evaluation," *arXiv preprint arXiv:1902.04491*, 2019.

[10] C. Rotsos, N. Sarrar, S. Uhlig, R. Sherwood, and A. W. Moore, "OFLOPS: An open framework for OpenFlow switch evaluation," in *Proceedings of the 13th International Conference on Passive and Active Measurement (PAM)*, 2012.

[11] D. Farrell, "WCBench: CBench, wrapped in stuff that makes it useful," https://github.com/dfarrell07/wcbench, 2015.

[12] A. Shalimov, D. Zuikov, D. Zimarina, V. Pashkov, and R. Smeliansky, "Advanced study of SDN/OpenFlow controllers," in *Proceedings of the 9th Central & Eastern European Software Engineering Conference (CEE-SECR)*, 2013.

[13] C. Popoviciu, A. Hamza, G. V. de Velde, and D. Dugatkin, "IPv6 benchmarking methodology for network interconnect devices," Internet Engineering Task Force, RFC 5180, 2008, IETF Benchmarking Methodology Working Group.

[14] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010.

[15] B. Xiong, K. Yang, J. Zhao, W. Li, and K. Li, "Performance evaluation of OpenFlow-based software-defined networks based on queueing model," *Computer Networks*, vol. 102, pp. 172–185, 2016.

[16] Open vSwitch, "OVS: Production quality, multilayer open virtual switch," https://www.openvswitch.org/, 2026, accessed: 2026-06-08.

[17] J. Dugan, S. Elliott, B. A. Mah, J. Poskanzer, and K. Prabhu, "iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool," https://software.es.net/iperf/, 2014.

[18] G. Rodola, "psutil: Cross-platform lib for process and system monitoring in Python," https://github.com/giampaolo/psutil, 2026, accessed: 2026-06-12.

[19] NTT Software Innovation Center, "Ryu SDN framework," https://github.com/osrg/ryu, 2012.

[20] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an open, distributed SDN OS," in *Proceedings of the Third Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014.